DATA STRUCTURES · WEEK 11 HOMEWORK

AVL Tree: rotations and rebalancing

Insert and delete step by step · BST to AVL · DFS / BFS traversals · Java code

Name _____ Student ID _____ Section _____

Rules used in every step

- **Height:** an empty subtree has height 0, a leaf has height 1.
- **Balance factor:** BF(n) = height(left subtree) − height(right subtree). It is the small number printed beside every node.
- **AVL condition:** every node has BF = −1, 0 or +1.
- **After an insert or delete** walk back up from the changed position to the root, recomputing heights and BFs. A node with |BF| = 2 is repaired with the rotation chosen from the table. After an insert one repair is always enough; after a delete keep checking up to the root.
- **Deleting a node with two children:** copy its *in-order successor* (the smallest key in its right subtree) into it, then delete the successor, which has at most one child.

Case	Unbalanced node z	Repair
LL	BF(z) = +2, left child BF ≥ 0	right rotation at z
RR	BF(z) = −2, right child BF ≤ 0	left rotation at z
LR	BF(z) = +2, left child BF = −1	left rotation at the left child, then right rotation at z
RL	BF(z) = −2, right child BF = +1	right rotation at the right child, then left rotation at z

new node · unbalanced (|BF| = 2) · moved by a rotation · node to delete · in-order successor · removed position · search path

1 Insert 90, then 12

The given tree (figure 1) is a valid AVL tree: every balance factor is −1, 0 or +1. In each "after the BST insert" diagram the balance factors are shown only up to the first unbalanced node — the nodes above it are checked after the repair.

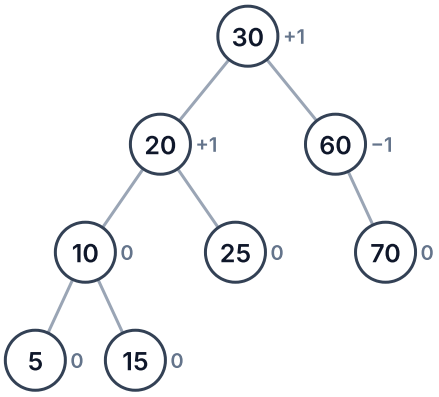


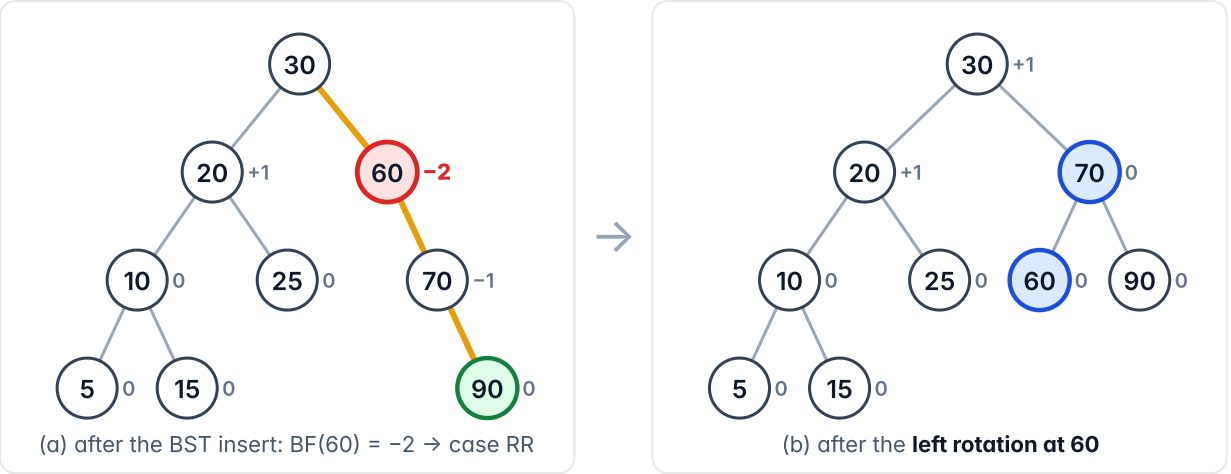
Figure 1 — given AVL tree, height 4

1.1 Insert 90

1. **BST insert.** 90 > 30 → go right; 90 > 60 → go right; 90 > 70 → go right. The right child of 70 is empty, so 90 is attached there.
2. **Recompute the balance factors on the way back up** (70 → 60 → 30): BF(70) = 0 − 1 = −1 ✓, BF(60) = 0 − 2 = **−2** ×. Node 60 is the first unbalanced node.
3. **Choose the case.** From 60 the new key went **R**ight and then **R**ight again (the right child 70 has BF −1) → **case RR** → single **left rotation at 60**: 70 moves up into the place of 60 and 60 becomes the left child of 70 (the old left

subtree of 70 — empty here — would become the right subtree of 60).

4. **Continue up.** BF(30) = 3 − 2 = +1 ✓. The tree is an AVL tree again.

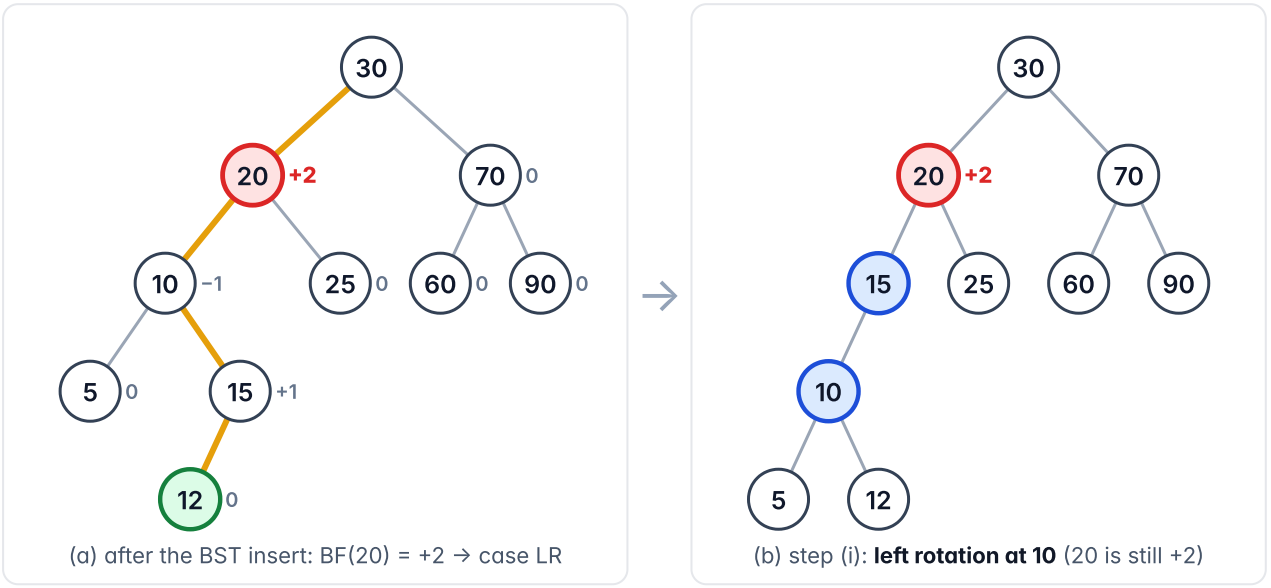


(a) after the BST insert: BF(60) = −2 → case RR

(b) after the **left rotation at 60**

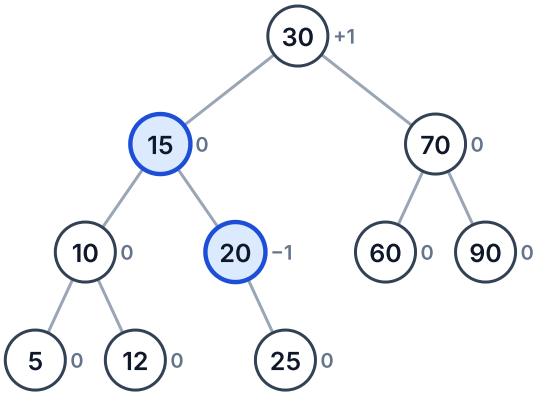
1.2 Insert 12 (into the tree from 1.1)

1. **BST insert.** 12 < 30 → left; 12 < 20 → left; 12 > 10 → right; 12 < 15 → left. The left child of 15 is empty, so 12 is attached there.
2. **Recompute the balance factors** (15 → 10 → 20 → 30): BF(15) = 1 − 0 = +1 ✓, BF(10) = 1 − 2 = −1 ✓, BF(20) = 3 − 1 = **+2** ✗. Node 20 is the first unbalanced node on the way up, so it is repaired first (30 is checked after the repair).
3. **Choose the case.** From 20 the new key went **L**eft and then **R**ight (the left child 10 has BF −1) → **case LR** → double rotation:
   (i) **left rotation at 10**: 15 moves up, 10 becomes the left child of 15 and the old left child of 15 (12) becomes the right child of 10 — the subtree now has the LL shape;
   (ii) **right rotation at 20**: 15 moves up into the place of 20 and 20 becomes the right child of 15.
4. **Continue up.** BF(30) = 3 − 2 = +1 ✓. Balanced.



(a) after the BST insert: BF(20) = +2 → case LR

(b) step (i): **left rotation at 10** (20 is still +2)



(c) step (ii): **right rotation at 20** — final tree

Answer, exercise 1: the final AVL tree is shown in (c). Rotations used: insert 90 → RR at 60 (left rotation); insert 12 → LR at 20 (left rotation at 10, then right rotation at 20). In-order: 5, 10, 12, 15, 20, 25, 30, 60, 70, 90 (still sorted, so the BST order is kept); height 4.

2 Delete 1, 32, 40, 50, 60 (in sequence)

Each deletion is applied to the tree left by the previous one. The given tree (figure 2) is a valid AVL tree.

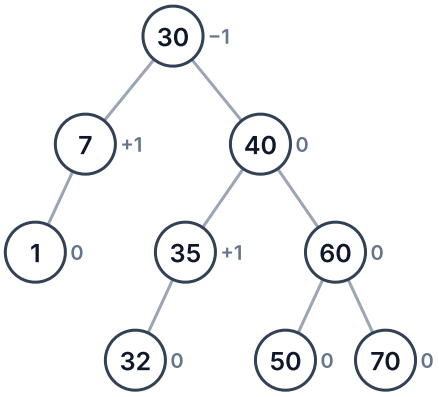
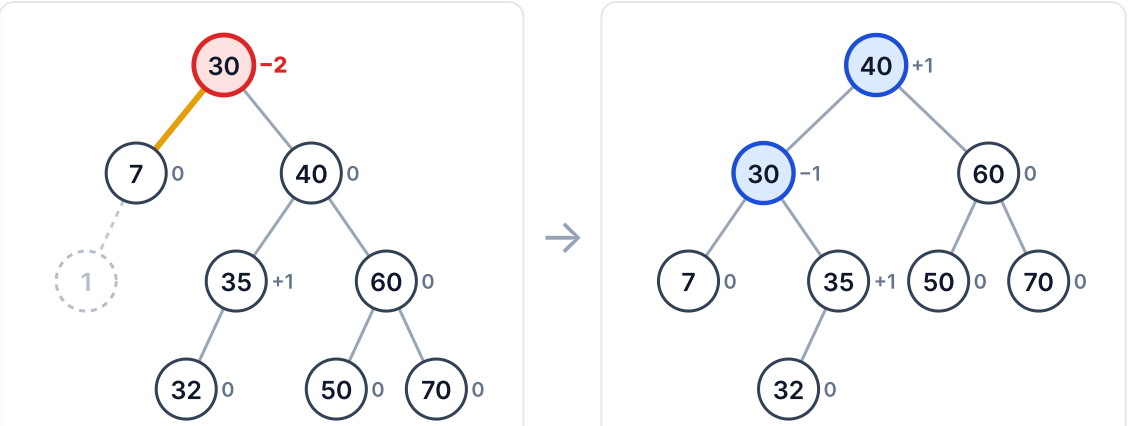


Figure 2 — given AVL tree, height 4

2.1 Delete 1

1. **Find it.** 1 < 30 → left; 1 < 7 → left; found. 1 is a **leaf**, so it is simply removed.
2. **Back up:** BF(7) = 0 − 0 = 0 ✓, BF(30) = 1 − 3 = **−2** ✗.
3. **Case.** The right child 40 has BF 0 → **case RR** (when deleting, a child with BF 0 is repaired with the single rotation) → **left rotation at 30**: 40 becomes the root, 30 becomes the left child of 40, and the old left subtree of 40 (35 with 32) becomes the right subtree of 30.
4. **Check:** BF(30) = 1 − 2 = −1 ✓, BF(40) = 3 − 2 = +1 ✓.

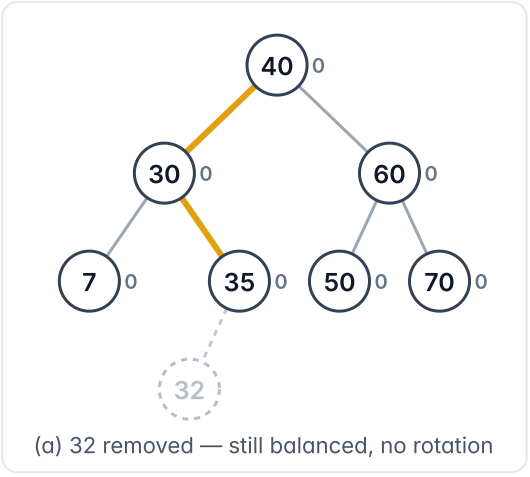


(a) 1 removed: BF(30) = −2 → case RR

(b) after the **left rotation at 30**

2.2 Delete 32

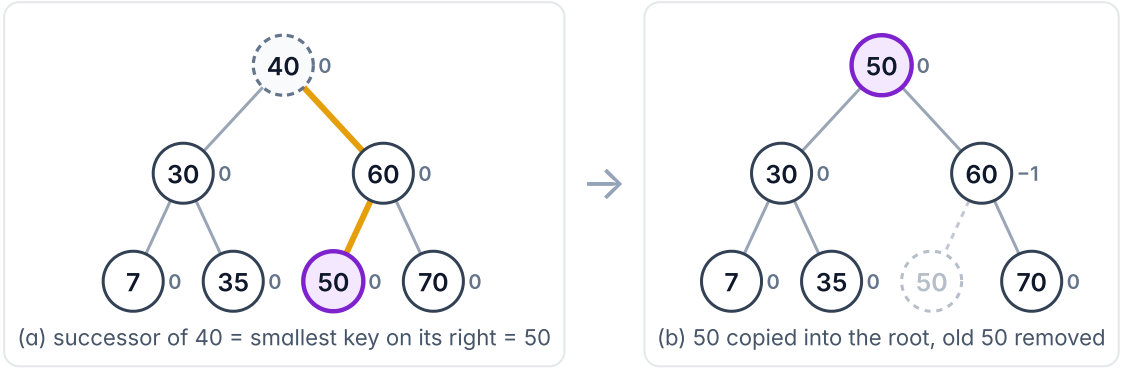
1. **Find it.** 32 < 40 → left; 32 > 30 → right; 32 < 35 → left; found a **leaf** → remove it.
2. **Back up:** BF(35) = 0, BF(30) = 1 − 1 = 0, BF(40) = 2 − 2 = 0 ✓ → **no rotation needed**.



(a) 32 removed — still balanced, no rotation

2.3 Delete 40

1. **Two children.** 40 (the root) has two children, so find its in-order successor: go right once (60), then left as far as possible (50). Successor = **50**.
2. **Replace.** Copy 50 into the root, then delete the old 50 from the right subtree — it is a leaf.
3. **Back up from 60:** BF(60) = 0 − 1 = −1 ✓, BF(50) = 2 − 2 = 0 ✓ → **no rotation needed**.

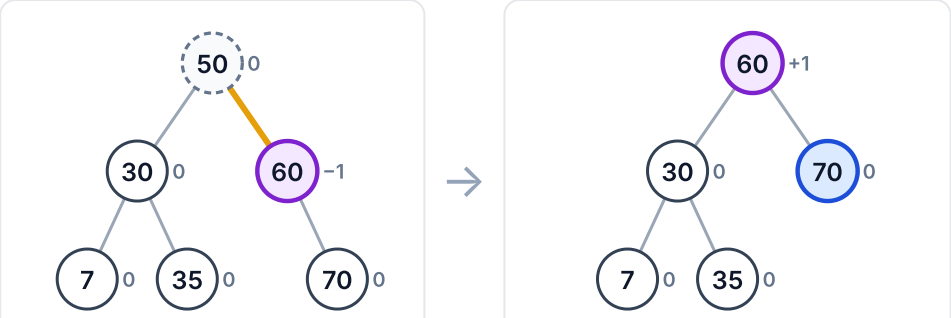


(a) successor of 40 = smallest key on its right = 50

(b) 50 copied into the root, old 50 removed

2.4 Delete 50

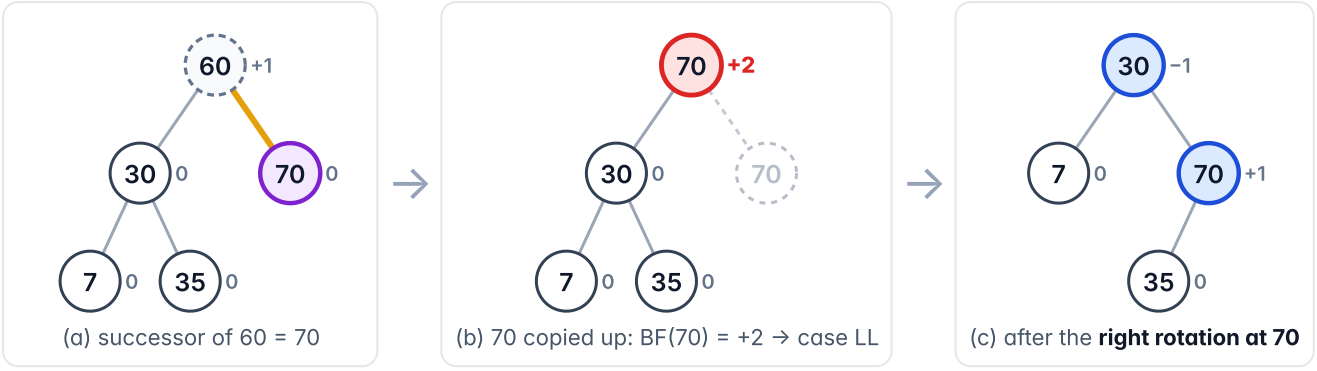
1. **Two children.** 50 (the root) has two children. Go right to 60; 60 has no left child, so the successor is **60**.
2. **Replace.** Copy 60 into the root, then delete the old 60. It has **one child** (70), so 70 takes its place.
3. **Back up:** BF(60) = 2 − 1 = +1 ✓ → **no rotation needed**.



(a) successor of 50 = 60 (b) 60 copied into the root; 70 moved up

2.5 Delete 60

1. **Two children.** 60 (the root) has two children. Its right child 70 has no left child, so the successor is **70**.
2. **Replace.** Copy 70 into the root and delete the old 70 (a leaf).
3. **Back up:** BF(70) = 2 − 0 = **+2** ×. The left child 30 has BF 0 → **case LL** → **right rotation at 70**: 30 becomes the root, 70 becomes the right child of 30, and the old right child of 30 (35) becomes the left child of 70.
4. **Check:** BF(70) = 1 − 0 = +1 ✓, BF(30) = 1 − 2 = −1 ✓.



(a) successor of 60 = 70 (b) 70 copied up: BF(70) = +2 → case LL (c) after the **right rotation at 70**

Delete	Node removed	Unbalanced node	Case	Rotation
1	leaf	30 (BF −2)	RR	left rotation at 30
32	leaf	–	–	none
40	two children → successor 50 copied up	–	–	none
50	two children → successor 60 copied up	–	–	none
60	two children → successor 70 copied up	70 (BF +2)	LL	right rotation at 70

Answer, exercise 2: the final AVL tree is 2.5 (c): root 30 with children 7 and 70, and 35 as the left child of 70. In-order: 7, 30, 35, 70; height 3.

3 Converting a BST into an AVL tree, and why AVL trees perform better

3.1 Are the given trees already AVL trees?

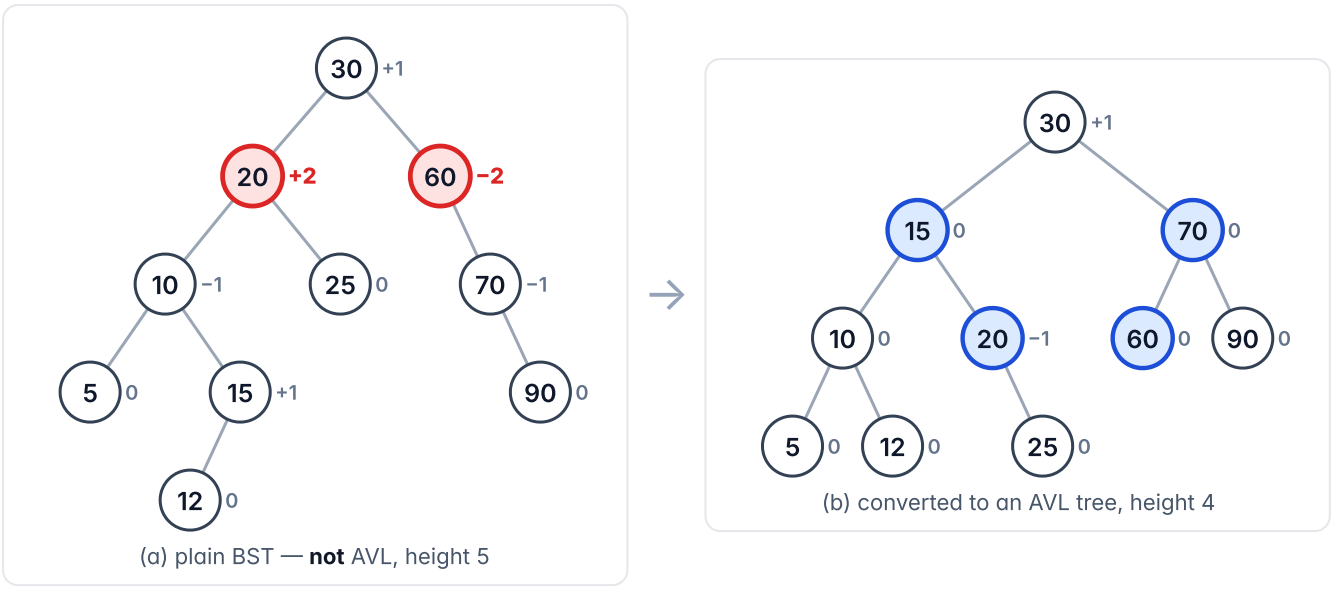
Yes. Figure 1: 5: 0, 10: 0, 15: 0, 20: +1, 25: 0, 30: +1, 60: −1, 70: 0. Figure 2: 1: 0, 7: +1, 30: −1, 32: 0, 35: +1, 40: 0, 50: 0, 60: 0, 70: 0. No node has $|BF| \geq 2$, so both trees are valid AVL trees as given. A tree stops being AVL only when keys are added or removed *without* rebalancing — that is when it has to be converted.

3.2 Converting a plain BST into an AVL tree

Insert 90 and 12 into figure 1 as an ordinary BST (no rotations). The result is still a valid BST but **not** an AVL tree: BF(60) = −2 and BF(20) = +2, height 5. To convert it, repair every node with $|BF| = 2$ from the bottom up with the same four cases:

- **20 is case LR** (left child 10 has BF −1) → left rotation at 10, then right rotation at 20;
- **60 is case RR** (right child 70 has BF −1) → left rotation at 60.

Afterwards every BF is −1, 0 or +1 and the height drops from 5 to 4 (the same tree as in exercise 1).

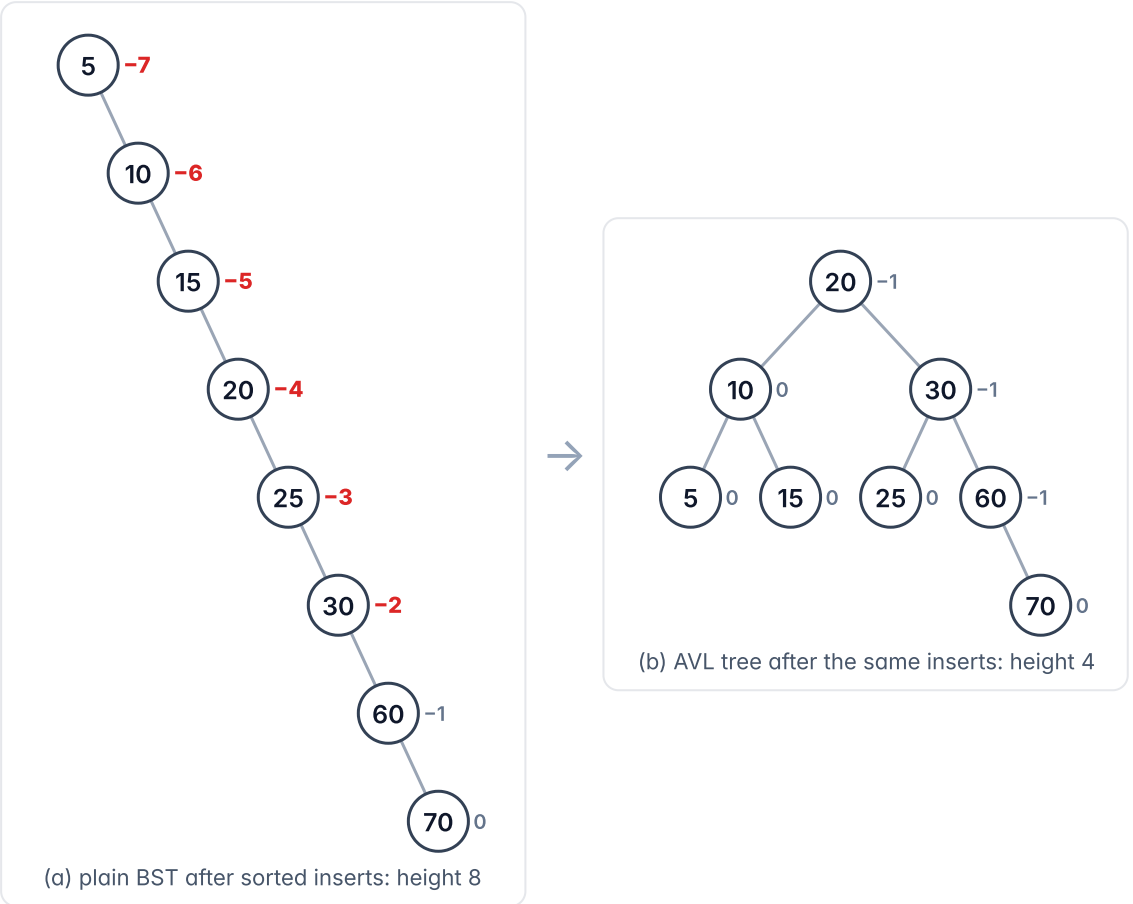


(a) plain BST — **not** AVL, height 5

(b) converted to an AVL tree, height 4

3.3 Worst case: keys arriving in sorted order

Insert 5, 10, 15, 20, 25, 30, 60, 70 in ascending order. A plain BST degenerates into a chain of height 8: finding 70 takes 8 comparisons, exactly like a linked list. An AVL tree does four RR rotations during the same inserts (at 5, 15, 10 and 25) and ends with height 4, so every key is found with at most 4 comparisons.



(a) plain BST after sorted inserts: height 8

(b) AVL tree after the same inserts: height 4

3.4 Performance advantages of an AVL tree compared with a BST

	BST, random order (average)	BST, worst case (e.g. sorted input)	AVL tree, worst case
Height with n keys	O(log n)	n	$< 1.44 \cdot \log_2(n + 2)$
Search	O(log n)	O(n)	**O(log n)**
Insert	O(log n)	O(n)	**O(log n)** + at most one single/double rotation
Delete	O(log n)	O(n)	**O(log n)** + at most O(log n) rotations, O(1) each
Depends on insertion order?	yes	yes — sorted input is the worst	**no**

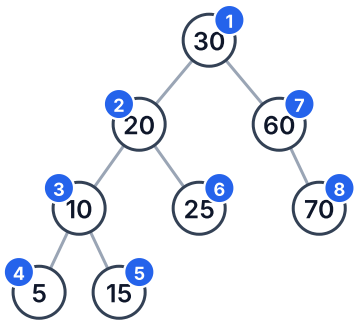
- **Guaranteed logarithmic height.** Rebalancing keeps the height below about $1.44 \cdot \log_2 n$, so search, insert and delete are O(log n) even in the worst case. A plain BST gives no guarantee: its shape depends on the insertion order and can degrade to a list, making every operation O(n).
- **Fast, predictable searching.** With a million keys an AVL tree needs at most 28 comparisons, a degenerate BST up to a million. AVL trees are very strictly balanced, which suits search-heavy work such as dictionaries, symbol tables and in-memory indexes.
- **Cheap repairs.** A rotation only changes a few links (O(1)) and keeps the in-order sequence, so the tree remains a valid BST. One insertion needs at most one single or double rotation.
- **The price.** Every node stores an extra height (or BF) field, and inserts and deletes do a little extra work to update heights and rotate. In exchange the performance never collapses, whatever the input order.

4 Tree traversals of the given figures (DFS and BFS)

Depth-first search (DFS) follows one branch as deep as possible before backing up (recursion, i.e. a stack). The three DFS orders differ only in *when* a node is visited: **pre-order** N L R (node before its subtrees), **in-order** L N R (between them) and **post-order** L R N (after them). **Breadth-first search (BFS)** visits the tree level by level, left to right, with a queue.

> How to read the working: each line expands every remaining call one level deeper — e.g. Pre(20) becomes [20, Pre(10), Pre(25)] — until only keys are left; newly visited keys are blue. The badge on each node in the diagrams is its visiting order.

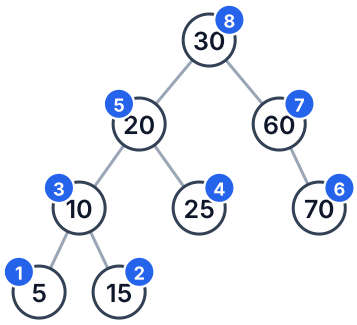
4.1 Figure 1



DFS: pre-order N → L → R

```
Pre(30) = 30, Pre(20), Pre(60)
= 30, [20, Pre(10), Pre(25)], [60, Pre(70)]
= 30, 20, [10, Pre(5), Pre(15)], 25, 60, 70
= 30, 20, 10, 5, 15, 25, 60, 70
```

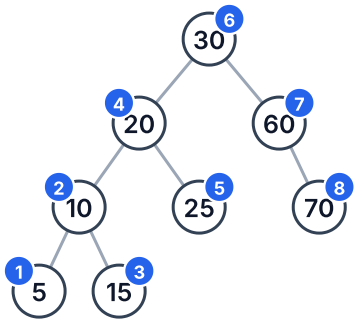
Visiting order: **30, 20, 10, 5, 15, 25, 60, 70**



DFS: post-order L → R → N

```
Post(30) = Post(20), Post(60), 30
= [Post(10), Post(25), 20], [Post(70), 60], 30
= [Post(5), Post(15), 10], 25, 20, 70, 60, 30
= 5, 15, 10, 25, 20, 70, 60, 30
```

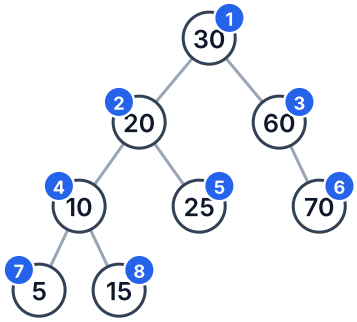
Visiting order: **5, 15, 10, 25, 20, 70, 60, 30**



DFS: in-order L → N → R

```
In(30) = In(20), 30, In(60)
= [In(10), 20, In(25)], 30, [60, In(70)]
= [In(5), 10, In(15)], 20, 25, 30, 60, 70
= 5, 10, 15, 20, 25, 30, 60, 70
```

Visiting order: **5, 10, 15, 20, 25, 30, 60, 70**

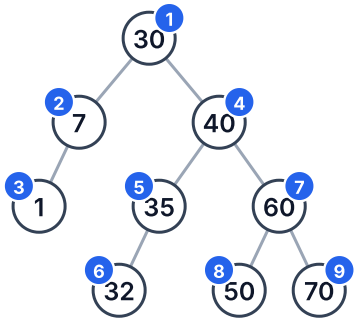


BFS: level order queue

Step	Dequeue & visit	Enqueue its children	Queue afterwards (front → back)
start	–	30	30
1	30	20, 60	20, 60
2	20	10, 25	60, 10, 25
3	60	70	10, 25, 70
4	10	5, 15	25, 70, 5, 15
5	25	–	70, 5, 15
6	70	–	5, 15
7	5	–	15
8	15	–	*empty*

Visiting order: **30, 20, 60, 10, 25, 70, 5, 15**

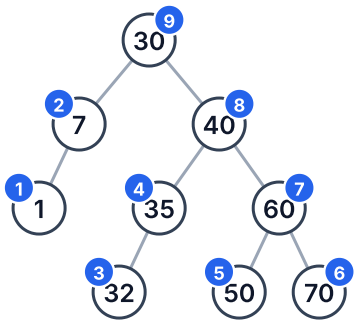
4.2 Figure 2



DFS: pre-order N → L → R

```
Pre(30) = 30, Pre(7), Pre(40)
= 30, [7, Pre(1)], [40, Pre(35), Pre(60)]
= 30, 7, 1, 40, [35, Pre(32)], [60, Pre(50), Pre(70)]
= 30, 7, 1, 40, 35, 32, 60, 50, 70
```

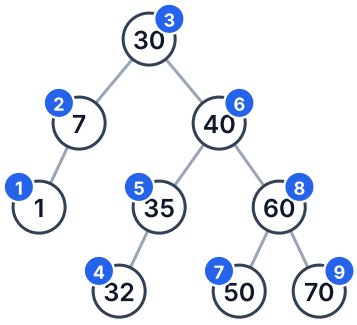
Visiting order: **30, 7, 1, 40, 35, 32, 60, 50, 70**



DFS: post-order L → R → N

```
Post(30) = Post(7), Post(40), 30
= [Post(1), 7], [Post(35), Post(60), 40], 30
= 1, 7, [Post(32), 35], [Post(50), Post(70), 60], 40, 30
= 1, 7, 32, 35, 50, 70, 60, 40, 30
```

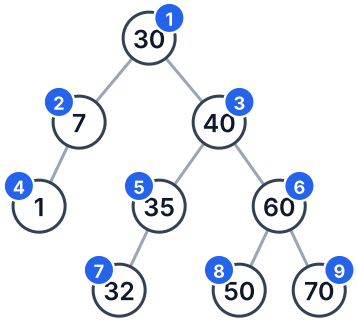
Visiting order: **1, 7, 32, 35, 50, 70, 60, 40, 30**



DFS: in-order L → N → R

```
In(30) = In(7), 30, In(40)
= [In(1), 7], 30, [In(35), 40, In(60)]
= 1, 7, 30, [In(32), 35], 40, [In(50), 60, In(70)]
= 1, 7, 30, 32, 35, 40, 50, 60, 70
```

Visiting order: **1, 7, 30, 32, 35, 40, 50, 60, 70**



BFS: level order queue

Step	Dequeue & visit	Enqueue its children	Queue afterwards (front → back)
start	–	30	30
1	30	7, 40	7, 40
2	7	1	40, 1
3	40	35, 60	1, 35, 60
4	1	–	35, 60
5	35	32	60, 32
6	60	50, 70	32, 50, 70
7	32	–	50, 70
8	50	–	70
9	70	–	*empty*

Visiting order: **30, 7, 40, 1, 35, 60, 32, 50, 70**

> In-order always lists the keys of a BST in ascending order — a quick check that both trees are valid binary search trees.

5 Coding test: AVL tree in Java

The program `AVLTree.java` (one file, JDK 8 or newer) builds an AVL tree with **insert** and **delete**, plus search and the four traversals. Each node stores its key, its height and links to its two children. The full source code is in the appendix.

Method	What it does
`insert(key)`	BST insert on the way down, then `rebalance()` on every node on the way back up
`delete(key)`	BST delete (leaf / one child / two children via the in-order successor), then `rebalance()` on the way back up
`rebalance(z)`	updates the height, computes BF, picks case LL / RR / LR / RL and performs the rotation(s)
`rotateLeft(x)`, `rotateRight(y)`	the two basic rotations; they also update the heights of the two nodes involved
`preOrder()`, `inOrder()`, `postOrder()`, `levelOrder()`	DFS traversals (recursive) and BFS (with a queue)
`isValidAVL()`	checks BST order, the stored heights and $\|BF\| \le 1$ for every node — printed as "AVL check: OK" after each operation
`draw(node)`	draws a tree or subtree top-down in plain ASCII, like the figures

How to compile and run

```
javac AVLTree.java
java AVLTree       # runs exercise 1 and exercise 2 and prints every step
java AVLTree -i    # interactive menu: insert / delete / search your own keys
```

For every operation the program prints the search path (Step 1); the height and balance factor of each node on the way back up, the rotation case and the subtree after every rotation (Step 2); and the resulting tree. The rotations it reports are exactly the ones worked out by hand in parts 1 and 2. The screenshots below are the program's complete output.

java AVLTree — part 1 of 5 - exercise 1: insert 90

```
$ javac AVLTree.java
$ java AVLTree
======================================================================
  AVL TREE - insert and delete with rebalancing      (Week 11 homework)
  BF(node) = height(left) - height(right), height of a leaf = 1
  a node with 2 children is deleted via its in-order successor
======================================================================

======================================================================
  EXERCISE 1 : insert 90, then 12, into the AVL tree of figure 1
======================================================================
Starting tree (figure 1):
                    30
              /-----+-----\
              20          60
         /----+----\      +--\
         10        25        70
      /--+--\
      5     15
  Pre-order   (N L R) : 30 20 10 5 15 25 60 70
  In-order    (L N R) : 5 10 15 20 25 30 60 70
  Post-order  (L R N) : 5 15 10 25 20 70 60 30
  Level-order (BFS)   : 30 20 60 10 25 70 5 15
  BF of each node     : 5(0) 10(0) 15(0) 20(+1) 25(0) 30(+1) 60(-1) 70(0)
  nodes = 8, height = 4, AVL check: OK

>>> INSERT 90
  Step 1 - BST insert: walk down from the root to an empty position
    at 30 : 90 > 30, go RIGHT
    at 60 : 90 > 60, go RIGHT
    at 70 : 90 > 70, go RIGHT
    empty position found: 90 becomes the RIGHT child of 70
  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2
    node 70 : h(L)=0  h(R)=1  BF=-1  ok
    node 60 : h(L)=0  h(R)=2  BF=-2  <-- UNBALANCED
        case RR (right child 70 has BF -1)
        fix : single LEFT rotation at 60
        LEFT rotation at 60: 70 moves up, 60 becomes its left child
        balanced subtree:
               70
           /--+--\
           60    90
    node 30 : h(L)=3  h(R)=2  BF=+1  ok
  Result - tree after inserting 90   (AVL check: OK, 1 rebalance)
                      30
              /------+------\
              20            70
         /----+----\     /--+--\
         10        25    60    90
      /--+--\
      5     15
```

Screen 1 of 5. Exercise 1 — compiling, the starting tree (figure 1) and **insert 90** (case RR at 60).

java AVLTree — part 2 of 5 - exercise 1: insert 12

```
>>> INSERT 12
  Step 1 - BST insert: walk down from the root to an empty position
    at 30 : 12 < 30, go LEFT
    at 20 : 12 < 20, go LEFT
    at 10 : 12 > 10, go RIGHT
    at 15 : 12 < 15, go LEFT
    empty position found: 12 becomes the LEFT child of 15
  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2
    node 15 : h(L)=1  h(R)=0  BF=+1  ok
    node 10 : h(L)=1  h(R)=2  BF=-1  ok
    node 20 : h(L)=3  h(R)=1  BF=+2  <-- UNBALANCED
        case LR (left child 10 has BF -1)
        fix : LEFT rotation at 10, then RIGHT rotation at 20
        (1) LEFT rotation at 10: 15 moves up, 10 becomes its left child, 12 moves to the right of 10
            subtree after (1):
                        20
                      /--+--\
                      15    25
                   /--+
                   10
                /--+--\
                5     12
        (2) RIGHT rotation at 20: 15 moves up, 20 becomes its right child
        balanced subtree:
                    15
                /----+----\
                10        20
             /--+--\      +--\
             5     12        25
    node 30 : h(L)=3  h(R)=2  BF=+1  ok
  Result - tree after inserting 12   (AVL check: OK, 1 rebalance)
                       30
              /--------+--------\
              15                70
         /----+----\         /--+--\
         10        20        60    90
      /--+--\      +--\
      5     12        25

Final tree of exercise 1 (drawn above):
  Pre-order   (N L R) : 30 15 10 5 12 20 25 70 60 90
  In-order    (L N R) : 5 10 12 15 20 25 30 60 70 90
  Post-order  (L R N) : 5 12 10 25 20 15 60 90 70 30
  Level-order (BFS)   : 30 15 70 10 20 60 90 5 12 25
  BF of each node     : 5(0) 10(0) 12(0) 15(0) 20(-1) 25(0) 30(+1) 60(0) 70(0) 90(0)
  nodes = 10, height = 4, AVL check: OK

Summary of exercise 1:
  insert 90  : RR at 60 -> left rotation at 60
  insert 12  : LR at 20 -> left rotation at 10, right rotation at 20
```

Screen 2 of 5. Exercise 1 — **insert 12** (case LR at 20: left rotation at 10, right rotation at 20), final tree and summary.

java AVLTree — part 3 of 5 - exercise 2: delete 1

```
======================================================================
  EXERCISE 2 : delete 1, 32, 40, 50, 60 (in sequence) from figure 2
======================================================================
Starting tree (figure 2):
              30
         /-----+-----\
         7           40
      /--+      /----+----\
      1         35        60
             /--+      /--+--\
             32        50    70
  Pre-order   (N L R) : 30 7 1 40 35 32 60 50 70
  In-order    (L N R) : 1 7 30 32 35 40 50 60 70
  Post-order  (L R N) : 1 7 32 35 50 70 60 40 30
  Level-order (BFS)   : 30 7 40 1 35 60 32 50 70
  BF of each node     : 1(0) 7(+1) 30(-1) 32(0) 35(+1) 40(0) 50(0) 60(0) 70(0)
  nodes = 9, height = 4, AVL check: OK

>>> DELETE 1
  Step 1 - BST delete: search for 1 starting at the root
    at 30 : 1 < 30, go LEFT
    at 7  : 1 < 7, go LEFT
    found 1: it is a leaf, so simply remove it
  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2
    node 7  : h(L)=0  h(R)=0  BF=0   ok
    node 30 : h(L)=1  h(R)=3  BF=-2  <-- UNBALANCED
        case RR (right child 40 has BF 0)
        fix : single LEFT rotation at 30
        LEFT rotation at 30: 40 moves up, 30 becomes its left child, 35 moves to the right of 30
        balanced subtree:
                        40
                 /------+------\
                 30            60
            /----+----\     /--+--\
            7         35    50    70
                   /--+
                   32
  Result - tree after deleting 1   (AVL check: OK, 1 rebalance)
                  40
           /------+------\
           30            60
      /----+----\     /--+--\
      7         35    50    70
             /--+
             32
```

Screen 3 of 5. Exercise 2 — the starting tree (figure 2) and **delete 1** (case RR at 30).

java AVLTree — part 4 of 5 - exercise 2: delete 32, 40, 50

```
>>> DELETE 32
  Step 1 - BST delete: search for 32 starting at the root
    at 40 : 32 < 40, go LEFT
    at 30 : 32 > 30, go RIGHT
    at 35 : 32 < 35, go LEFT
    found 32: it is a leaf, so simply remove it
  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2
    node 35 : h(L)=0  h(R)=0  BF=0   ok
    node 30 : h(L)=1  h(R)=1  BF=0   ok
    node 40 : h(L)=2  h(R)=2  BF=0   ok
  Result - tree after deleting 32   (AVL check: OK, no rotation)
              40
        /-----+-----\
        30          60
     /--+--\     /--+--\
     7     35    50    70

>>> DELETE 40
  Step 1 - BST delete: search for 40 starting at the root
    found 40: it has two children, so use its in-order successor
    successor = smallest key in the right subtree of 40 = 50
    copy 50 into this node, then delete the old 50 from the right subtree
    at 60 : 50 < 60, go LEFT
    found 50: it is a leaf, so simply remove it
  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2
    node 60 : h(L)=0  h(R)=1  BF=-1  ok
    node 50 : h(L)=2  h(R)=2  BF=0   ok
  Result - tree after deleting 40   (AVL check: OK, no rotation)
             50
        /----+----\
        30        60
     /--+--\      +--\
     7     35        70

>>> DELETE 50
  Step 1 - BST delete: search for 50 starting at the root
    found 50: it has two children, so use its in-order successor
    successor = smallest key in the right subtree of 50 = 60
    copy 60 into this node, then delete the old 60 from the right subtree
    found 60: it has one child, so 70 takes its place
  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2
    node 60 : h(L)=2  h(R)=1  BF=+1  ok
  Result - tree after deleting 50   (AVL check: OK, no rotation)
             60
        /----+----\
        30        70
     /--+--\
     7     35
```

Screen 4 of 5. Exercise 2 — **delete 32** (leaf), **delete 40** and **delete 50** (two children, in-order successor).

```
java AVLTree — part 5 of 5 - exercise 2: delete 60

>>> DELETE 60
  Step 1 - BST delete: search for 60 starting at the root
    found 60: it has two children, so use its in-order successor
    successor = smallest key in the right subtree of 60 = 70
    copy 70 into this node, then delete the old 70 from the right subtree
    found 70: it is a leaf, so simply remove it
  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2
    node 70 : h(L)=2  h(R)=0  BF=+2  <-- UNBALANCED
        case LL (left child 30 has BF 0)
        fix : single RIGHT rotation at 70
        RIGHT rotation at 70: 30 moves up, 70 becomes its right child, 35 moves to the left of 70
        balanced subtree:
                 30
             /----+----\
             7         70
                    /--+
                    35
  Result - tree after deleting 60   (AVL check: OK, 1 rebalance)
            30
       /----+----\
       7         70
              /--+
              35

Final tree of exercise 2 (drawn above):
  Pre-order   (N L R) : 30 7 70 35
  In-order    (L N R) : 7 30 35 70
  Post-order  (L R N) : 7 35 70 30
  Level-order (BFS)   : 30 7 70 35
  BF of each node     : 7(0) 30(-1) 35(0) 70(+1)
  nodes = 4, height = 3, AVL check: OK

Summary of exercise 2:
  delete 1   : RR at 30 -> left rotation at 30
  delete 32  : no rotation needed
  delete 40  : no rotation needed   [2 children, successor 50 copied up]
  delete 50  : no rotation needed   [2 children, successor 60 copied up]
  delete 60  : LL at 70 -> right rotation at 70   [2 children, successor 70 copied up]

======================================================================
  Run  java AVLTree -i  to insert / delete your own keys.
======================================================================
```

Screen 5 of 5. Exercise 2 — **delete 60** (case LL at 70), final tree and summary of all rotations.

A Appendix: source code — AVLTree.java

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

/**
 * Week 11 Homework - AVL Tree (a self-balancing binary search tree).
 *
 * Operations: insert, delete, search and the four traversals
 * (pre-order, in-order, post-order and level-order / BFS).
 *
 * Every insert and delete prints what the algorithm does, step by step:
 *   Step 1  the BST search path from the root down to the insert/delete position
 *   Step 2  walking back up to the root: the new height and balance factor of
 *           every node, the rotation case (LL, RR, LR, RL) of an unbalanced
 *           node and the subtree after every rotation
 *   Result  the whole tree, drawn top-down like the figures in class
 *
 * Conventions (the same as in the written solution):
 *   height(empty tree) = 0, height(leaf) = 1
 *   BF(node) = height(left subtree) - height(right subtree)
 *   AVL property: every node has BF = -1, 0 or +1
 *   A node with two children is deleted by copying its in-order successor
 *   (the smallest key in its right subtree) into it and then deleting the
 *   successor from the right subtree.
 *
 * Compile and run (JDK 8 or newer):
 *   javac AVLTree.java
 *   java AVLTree       runs Exercise 1 (insert) and Exercise 2 (delete)
 *   java AVLTree -i    interactive menu: insert / delete / search / show
 */
public class AVLTree {

    /** The two trees of the homework, listed level by level (BFS order). */
    static final int[] FIGURE_1 = {30, 20, 60, 10, 25, 70, 5, 15};
    static final int[] FIGURE_2 = {30, 7, 40, 1, 35, 60, 32, 50, 70};

    /** One node of the tree. */
    static final class Node {
        int key;
        int height = 1;
        Node left;
        Node right;

        Node(int key) {
            this.key = key;
        }
    }

    private static final String RULE =
            "======================================================================";

    private Node root;
    private boolean trace = true;                                  // print the steps?
    private final List<String> rotations = new ArrayList<>();      // rotations of the current operation
    private String note = "";                                      // remark about the current operation
    private final List<String> history = new ArrayList<>();        // one summary line per operation

    // ======================================================================
    //  Small helpers
    // ======================================================================

    private static int height(Node n) {
        return n == null ? 0 : n.height;
    }

    private static int balanceFactor(Node n) {
        return n == null ? 0 : height(n.left) - height(n.right);
    }

```

```
    private static void updateHeight(Node n) {
        n.height = 1 + Math.max(height(n.left), height(n.right));
    }

    private static String signed(int value) {
        return value > 0 ? "+" + value : String.valueOf(value);
    }

    private void say(String line) {
        if (trace) {
            System.out.println(line);
        }
    }

    // ================================================================
    //  Rotations
    //
    //   Right rotation at y                Left rotation at x
    //
    //        y              x                  x                  y
    //       / \            / \                / \                / \
    //      x   C   ==>    A   y              A   y      ==>     x   C
    //     / \                / \                / \            / \
    //    A   B              B   C              B   C          A   B
    // ================================================================

    private Node rotateRight(Node y, String prefix) {
        Node x = y.left;
        Node b = x.right;
        x.right = y;
        y.left = b;
        updateHeight(y);
        updateHeight(x);
        say(prefix + "RIGHT rotation at " + y.key + ": " + x.key + " moves up, "
                + y.key + " becomes its right child"
                + (b == null ? "" : ", " + b.key + " moves to the left of " + y.key));
        return x;
    }

    private Node rotateLeft(Node x, String prefix) {
        Node y = x.right;
        Node b = y.left;
        y.left = x;
        x.right = b;
        updateHeight(x);
        updateHeight(y);
        say(prefix + "LEFT rotation at " + x.key + ": " + y.key + " moves up, "
                + x.key + " becomes its left child"
                + (b == null ? "" : ", " + b.key + " moves to the right of " + x.key));
        return y;
    }

    // ================================================================
    //  Rebalancing - used by both insert and delete
    // ================================================================

    /**
     * Runs for every node on the way back up to the root: updates the height,
     * prints the balance factor and, when |BF| = 2, repairs the node with one
     * or two rotations. Returns the root of this subtree after the repair.
     */
    private Node rebalance(Node z) {
        updateHeight(z);
        int bf = balanceFactor(z);
        boolean balanced = Math.abs(bf) <= 1;
        say(String.format("    node %-3d: h(L)=%d  h(R)=%d  BF=%-3s %s", z.key,
                height(z.left), height(z.right), signed(bf), balanced ? "ok" : "<-- UNBALANCED"));
        if (balanced) {
            return z;
        }

        String pad = "        ";
        Node newRoot;
        if (bf > 1) {                                   // the left side is too tall
            Node y = z.left;
            int childBf = balanceFactor(y);
            if (childBf >= 0) {
```

```
                say(pad + "case LL (left child " + y.key + " has BF " + signed(childBf) + ")");
                say(pad + "fix : single RIGHT rotation at " + z.key);
                newRoot = rotateRight(z, pad);
                rotations.add("LL at " + z.key + " -> right rotation at " + z.key);
            } else {
                say(pad + "case LR (left child " + y.key + " has BF " + signed(childBf) + ")");
                say(pad + "fix : LEFT rotation at " + y.key + ", then RIGHT rotation at " + z.key);
                z.left = rotateLeft(y, pad + "(1) ");
                printSubtree(pad + "    subtree after (1):", z);
                newRoot = rotateRight(z, pad + "(2) ");
                rotations.add("LR at " + z.key + " -> left rotation at " + y.key
                        + ", right rotation at " + z.key);
            }
        } else {                                        // the right side is too tall
            Node y = z.right;
            int childBf = balanceFactor(y);
            if (childBf <= 0) {
                say(pad + "case RR (right child " + y.key + " has BF " + signed(childBf) + ")");
                say(pad + "fix : single LEFT rotation at " + z.key);
                newRoot = rotateLeft(z, pad);
                rotations.add("RR at " + z.key + " -> left rotation at " + z.key);
            } else {
                say(pad + "case RL (right child " + y.key + " has BF " + signed(childBf) + ")");
                say(pad + "fix : RIGHT rotation at " + y.key + ", then LEFT rotation at " + z.key);
                z.right = rotateRight(y, pad + "(1) ");
                printSubtree(pad + "    subtree after (1):", z);
                newRoot = rotateLeft(z, pad + "(2) ");
                rotations.add("RL at " + z.key + " -> right rotation at " + y.key
                        + ", left rotation at " + z.key);
            }
        }
        printSubtree(pad + "balanced subtree:", newRoot);
        return newRoot;
    }

    // ======================================================================
    //  Insert
    // ======================================================================

    /** Inserts a key (duplicates are ignored) and prints every step. */
    public void insert(int key) {
        System.out.println();
        System.out.println(">>> INSERT " + key);
        if (contains(key)) {
            System.out.println("    " + key + " is already in the tree (no duplicates) - nothing to do");
            return;
        }
        rotations.clear();
        note = "";
        say("  Step 1 - BST insert: walk down from the root to an empty position");
        root = insert(root, key, null, "");
        finish("insert " + key, "inserting " + key);
    }

    private Node insert(Node node, int key, Node parent, String side) {
        if (node == null) {
            if (parent == null) {
                say("    the tree is empty, so " + key + " becomes the root");
            } else {
                say("    empty position found: " + key + " becomes the " + side + " child of " + parent.key);
                say("  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2");
            }
            return new Node(key);
        }
        if (key < node.key) {
            say(String.format("    at %-3d: %d < %d, go LEFT", node.key, key, node.key));
            node.left = insert(node.left, key, node, "LEFT");
        } else if (key > node.key) {
            say(String.format("    at %-3d: %d > %d, go RIGHT", node.key, key, node.key));
            node.right = insert(node.right, key, node, "RIGHT");
        } else {
            return node;                                // duplicate key: nothing changes
        }
        return rebalance(node);
    }

    // ======================================================================
```

```
    //  Delete
    // ========================================================================

    /** Deletes a key and prints every step. */
    public void delete(int key) {
        System.out.println();
        System.out.println(">>> DELETE " + key);
        if (!contains(key)) {
            System.out.println("    " + key + " is not in the tree - nothing to delete");
            return;
        }
        rotations.clear();
        note = "";
        say("  Step 1 - BST delete: search for " + key + " starting at the root");
        root = delete(root, key);
        finish("delete " + key, "deleting " + key);
    }

    private Node delete(Node node, int key) {
        if (key < node.key) {
            say(String.format("    at %-3d: %d < %d, go LEFT", node.key, key, node.key));
            node.left = delete(node.left, key);
        } else if (key > node.key) {
            say(String.format("    at %-3d: %d > %d, go RIGHT", node.key, key, node.key));
            node.right = delete(node.right, key);
        } else if (node.left == null || node.right == null) {
            Node child = node.left != null ? node.left : node.right;
            if (child == null) {
                say("    found " + key + ": it is a leaf, so simply remove it");
            } else {
                say("    found " + key + ": it has one child, so " + child.key + " takes its place");
            }
            say("  Step 2 - walk back up to the root: update height, check BF, rotate if |BF| = 2");
            return child;                                  // this subtree did not change: still AVL
        } else {
            Node successor = node.right;
            while (successor.left != null) {
                successor = successor.left;
            }
            say("    found " + key + ": it has two children, so use its in-order successor");
            say("    successor = smallest key in the right subtree of " + key + " = " + successor.key);
            say("    copy " + successor.key + " into this node, then delete the old "
                    + successor.key + " from the right subtree");
            note = "2 children, successor " + successor.key + " copied up";
            node.key = successor.key;
            node.right = delete(node.right, successor.key);
        }
        return rebalance(node);
    }

    /** Prints the result of an insert/delete and remembers it for the summary. */
    private void finish(String operation, String verb) {
        String what = rotations.isEmpty() ? "no rotation needed" : String.join("; ", rotations);
        if (!note.isEmpty()) {
            what += "   [" + note + "]";
        }
        history.add(String.format("%-10s : %s", operation, what));
        System.out.println("  Result - tree after " + verb + "   (AVL check: "
                + (isValidAVL() ? "OK" : "FAILED") + ", "
                + (rotations.isEmpty() ? "no rotation" : rotations.size() + " rebalance") + ")");
        printTree(root, "      ");
    }

    // ========================================================================
    //  Search, traversals, checks
    // ========================================================================

    public boolean contains(int key) {
        Node n = root;
        while (n != null) {
            if (key == n.key) {
                return true;
            }
            n = key < n.key ? n.left : n.right;
        }
        return false;
    }
```

```
305
306     /** Prints the search path for a key. */
307     public void search(int key) {
308         StringBuilder path = new StringBuilder();
309         Node n = root;
310         int comparisons = 0;
311         while (n != null) {
312             comparisons++;
313             path.append(n.key);
314             if (key == n.key) {
315                 System.out.println("    found " + key + " after " + comparisons + " comparison(s), path: " +
    path);
316                 return;
317             }
318             path.append(key < n.key ? " -L-> " : " -R-> ");
319             n = key < n.key ? n.left : n.right;
320         }
321         System.out.println("    " + key + " is NOT in the tree, path: " + path + "null");
322     }
323
324     public List<Integer> preOrder() {
325         List<Integer> out = new ArrayList<>();
326         preOrder(root, out);
327         return out;
328     }
329
330     private static void preOrder(Node n, List<Integer> out) {      // Node, Left, Right
331         if (n != null) {
332             out.add(n.key);
333             preOrder(n.left, out);
334             preOrder(n.right, out);
335         }
336     }
337
338     public List<Integer> inOrder() {
339         List<Integer> out = new ArrayList<>();
340         inOrder(root, out);
341         return out;
342     }
343
344     private static void inOrder(Node n, List<Integer> out) {       // Left, Node, Right
345         if (n != null) {
346             inOrder(n.left, out);
347             out.add(n.key);
348             inOrder(n.right, out);
349         }
350     }
351
352     public List<Integer> postOrder() {
353         List<Integer> out = new ArrayList<>();
354         postOrder(root, out);
355         return out;
356     }
357
358     private static void postOrder(Node n, List<Integer> out) {     // Left, Right, Node
359         if (n != null) {
360             postOrder(n.left, out);
361             postOrder(n.right, out);
362             out.add(n.key);
363         }
364     }
365
366     /** Breadth-first search: visit the tree level by level using a queue. */
367     public List<Integer> levelOrder() {
368         List<Integer> out = new ArrayList<>();
369         ArrayDeque<Node> queue = new ArrayDeque<>();
370         if (root != null) {
371             queue.add(root);
372         }
373         while (!queue.isEmpty()) {
374             Node n = queue.poll();
375             out.add(n.key);
376             if (n.left != null) {
377                 queue.add(n.left);
378             }
379             if (n.right != null) {
380                 queue.add(n.right);
```

```
            }
        }
        return out;
    }

    /** True when the keys are in BST order, every stored height is right and every |BF| <= 1. */
    public boolean isValidAVL() {
        return check(root, Long.MIN_VALUE, Long.MAX_VALUE) >= 0;
    }

    private static int check(Node n, long low, long high) {        // returns the height, or -1 if invalid
        if (n == null) {
            return 0;
        }
        if (n.key <= low || n.key >= high) {
            return -1;
        }
        int hl = check(n.left, low, n.key);
        int hr = check(n.right, n.key, high);
        if (hl < 0 || hr < 0 || Math.abs(hl - hr) > 1 || n.height != 1 + Math.max(hl, hr)) {
            return -1;
        }
        return n.height;
    }

    private static String join(List<Integer> keys) {
        StringBuilder sb = new StringBuilder();
        for (int k : keys) {
            sb.append(sb.length() == 0 ? "" : " ").append(k);
        }
        return sb.toString();
    }

    private String balanceFactors() {
        StringBuilder sb = new StringBuilder();
        appendBalanceFactors(root, sb);
        return sb.toString().trim();
    }

    private static void appendBalanceFactors(Node n, StringBuilder sb) {
        if (n != null) {
            appendBalanceFactors(n.left, sb);
            sb.append(n.key).append('(').append(signed(balanceFactor(n))).append(") ");
            appendBalanceFactors(n.right, sb);
        }
    }

    /** Prints the traversals, balance factors and size of the tree. */
    public void printDetails() {
        System.out.println("  Pre-order   (N L R) : " + join(preOrder()));
        System.out.println("  In-order    (L N R) : " + join(inOrder()));
        System.out.println("  Post-order  (L R N) : " + join(postOrder()));
        System.out.println("  Level-order (BFS)   : " + join(levelOrder()));
        System.out.println("  BF of each node     : " + balanceFactors());
        System.out.println("  nodes = " + inOrder().size() + ", height = " + height(root)
                + ", AVL check: " + (isValidAVL() ? "OK" : "FAILED"));
    }

    public void show() {
        printTree(root, "      ");
        printDetails();
    }

    private void printHistory(String title) {
        System.out.println();
        System.out.println(title + ":");
        for (String line : history) {
            System.out.println("  " + line);
        }
    }

    /** Builds a tree silently (no trace), e.g. from the level order of a figure. */
    static AVLTree build(int... keys) {
        AVLTree tree = new AVLTree();
        tree.trace = false;
        for (int key : keys) {
            tree.root = tree.insert(tree.root, key, null, "");
```

```
        }
        tree.rotations.clear();
        tree.trace = true;
        return tree;
    }

    // ====================================================================
    //  Drawing the tree top-down in plain ASCII (works in every console)
    //
    //  Each node gets an x position: a parent is centred above its children
    //  and two neighbouring subtrees are pushed apart until they no longer
    //  overlap on any level (connectors included). Then every node is printed
    //  on row 2*level and its "/---+---\" connector on the row below it.
    // ====================================================================

    /** x position of every node (relative to the subtree root) and the outline of each level. */
    private static final class Layout {
        final Map<Node, Integer> x = new HashMap<>();
        final List<int[]> outline = new ArrayList<>();          // outline.get(level) = {min x, max x}
    }

    private static Layout layout(Node n, int gap) {
        if (n == null) {
            return null;
        }
        Layout left = layout(n.left, gap);
        Layout right = layout(n.right, gap);
        int leftOffset = -gap / 2;
        int rightOffset = gap / 2;
        if (left != null && right != null) {
            int distance = gap;
            int levels = Math.min(left.outline.size(), right.outline.size());
            for (int d = 0; d < levels; d++) {
                distance = Math.max(distance, left.outline.get(d)[1] - right.outline.get(d)[0] + gap);
            }
            distance += distance % 2;                           // keep the parent exactly in the middle
            leftOffset = -distance / 2;
            rightOffset = distance / 2;
        }
        Layout result = new Layout();
        result.x.put(n, 0);
        // the outline of a level also covers the "/---+---\" connector below each node,
        // so a neighbouring node never sits above (or touches) someone else's connector
        result.outline.add(new int[] {left != null ? leftOffset : 0, right != null ? rightOffset : 0});
        attach(result, left, leftOffset);
        attach(result, right, rightOffset);
        return result;
    }

    private static void attach(Layout parent, Layout child, int offset) {
        if (child == null) {
            return;
        }
        for (Map.Entry<Node, Integer> e : child.x.entrySet()) {
            parent.x.put(e.getKey(), e.getValue() + offset);
        }
        for (int d = 0; d < child.outline.size(); d++) {
            int low = child.outline.get(d)[0] + offset;
            int high = child.outline.get(d)[1] + offset;
            if (d + 1 < parent.outline.size()) {
                int[] o = parent.outline.get(d + 1);
                o[0] = Math.min(o[0], low);
                o[1] = Math.max(o[1], high);
            } else {
                parent.outline.add(new int[] {low, high});
            }
        }
    }

    private static int levels(Node n) {
        return n == null ? 0 : 1 + Math.max(levels(n.left), levels(n.right));
    }

    private static int widestKey(Node n) {
        return n == null ? 0
                : Math.max(String.valueOf(n.key).length(), Math.max(widestKey(n.left), widestKey(n.right)));
    }
```

```
535
536     /** Returns the drawing of the subtree rooted at n, one string per line. */
537     static List<String> draw(Node n) {
538         List<String> lines = new ArrayList<>();
539         if (n == null) {
540             lines.add("(empty tree)");
541             return lines;
542         }
543         int keyWidth = widestKey(n);
544         Layout layout = layout(n, Math.max(6, keyWidth + 4));
545         int minX = 0;
546         int maxX = 0;
547         for (int[] o : layout.outline) {
548             minX = Math.min(minX, o[0]);
549             maxX = Math.max(maxX, o[1]);
550         }
551         int shift = -minX + (keyWidth - 1) / 2;
552         char[][] grid = new char[2 * levels(n) - 1][maxX + shift + keyWidth + 1];
553         for (char[] row : grid) {
554             Arrays.fill(row, ' ');
555         }
556         paint(n, 0, layout, shift, grid);
557         for (char[] row : grid) {
558             lines.add(new String(row).replaceAll("\\s+$", ""));
559         }
560         return lines;
561     }
562
563     private static void paint(Node n, int level, Layout layout, int shift, char[][] grid) {
564         int c = layout.x.get(n) + shift;
565         String label = String.valueOf(n.key);
566         int start = c - (label.length() - 1) / 2;
567         for (int i = 0; i < label.length(); i++) {
568             grid[2 * level][start + i] = label.charAt(i);
569         }
570         if (n.left == null && n.right == null) {
571             return;
572         }
573         char[] row = grid[2 * level + 1];
574         int from = n.left != null ? layout.x.get(n.left) + shift : c;
575         int to = n.right != null ? layout.x.get(n.right) + shift : c;
576         for (int i = from; i <= to; i++) {
577             row[i] = '-';
578         }
579         if (n.left != null) {
580             row[from] = '/';
581             paint(n.left, level + 1, layout, shift, grid);
582         }
583         if (n.right != null) {
584             row[to] = '\\';
585             paint(n.right, level + 1, layout, shift, grid);
586         }
587         row[c] = '+';
588     }
589
590     private static void printTree(Node n, String indent) {
591         for (String line : draw(n)) {
592             System.out.println(indent + line);
593         }
594     }
595
596     private void printSubtree(String title, Node n) {
597         if (trace) {
598             System.out.println(title);
599             printTree(n, title.replaceAll("\\S.*$", "") + "    ");
600         }
601     }
602
603     // ====================================================================
604     //  Main program
605     // ====================================================================
606
607     public static void main(String[] args) {
608         if (args.length > 0 && (args[0].equals("-i") || args[0].equals("--interactive"))) {
609             interactive();
610             return;
611         }
```

```
        System.out.println(RULE);
        System.out.println("  AVL TREE - insert and delete with rebalancing      (Week 11 homework)");
        System.out.println("  BF(node) = height(left) - height(right), height of a leaf = 1");
        System.out.println("  a node with 2 children is deleted via its in-order successor");
        System.out.println(RULE);

        heading("EXERCISE 1 : insert 90, then 12, into the AVL tree of figure 1");
        AVLTree tree1 = build(FIGURE_1);
        System.out.println("Starting tree (figure 1):");
        tree1.show();
        tree1.insert(90);
        tree1.insert(12);
        System.out.println();
        System.out.println("Final tree of exercise 1 (drawn above):");
        tree1.printDetails();
        tree1.printHistory("Summary of exercise 1");

        heading("EXERCISE 2 : delete 1, 32, 40, 50, 60 (in sequence) from figure 2");
        AVLTree tree2 = build(FIGURE_2);
        System.out.println("Starting tree (figure 2):");
        tree2.show();
        for (int key : new int[] {1, 32, 40, 50, 60}) {
            tree2.delete(key);
        }
        System.out.println();
        System.out.println("Final tree of exercise 2 (drawn above):");
        tree2.printDetails();
        tree2.printHistory("Summary of exercise 2");
        System.out.println();
        System.out.println(RULE);
        System.out.println("  Run  java AVLTree -i  to insert / delete your own keys.");
        System.out.println(RULE);
    }

    private static void heading(String title) {
        System.out.println();
        System.out.println(RULE);
        System.out.println("  " + title);
        System.out.println(RULE);
    }

    /** Simple menu so any key can be inserted, deleted or searched. */
    private static void interactive() {
        Scanner in = new Scanner(System.in);
        AVLTree tree = new AVLTree();
        System.out.println("AVL TREE - interactive mode (keys are integers, several keys may be typed at
once)");
        while (true) {
            System.out.println();
            System.out.println("1) Insert  2) Delete  3) Search  4) Show tree  5) Load figure 1  6) Load figure
2");
            System.out.println("7) Clear   8) Step-by-step trace on/off (now " + (tree.trace ? "ON" : "OFF") +
")  0) Exit");
            System.out.print("Choose: ");
            if (!in.hasNextLine()) {
                return;
            }
            String choice = in.nextLine().trim();
            switch (choice) {
                case "1":
                    for (int key : readKeys(in, "Key(s) to insert: ")) {
                        tree.insert(key);
                    }
                    break;
                case "2":
                    for (int key : readKeys(in, "Key(s) to delete: ")) {
                        tree.delete(key);
                    }
                    break;
                case "3":
                    for (int key : readKeys(in, "Key(s) to search: ")) {
                        tree.search(key);
                    }
                    break;
                case "4":
                    tree.show();
                    break;
```

```
                case "5":
                case "6":
                    boolean keepTrace = tree.trace;
                    tree = build(choice.equals("5") ? FIGURE_1 : FIGURE_2);
                    tree.trace = keepTrace;
                    System.out.println("Loaded figure " + (choice.equals("5") ? 1 : 2) + ":");
                    tree.show();
                    break;
                case "7":
                    boolean trace = tree.trace;
                    tree = new AVLTree();
                    tree.trace = trace;
                    System.out.println("The tree is now empty.");
                    break;
                case "8":
                    tree.trace = !tree.trace;
                    System.out.println("Step-by-step trace is now " + (tree.trace ? "ON" : "OFF"));
                    break;
                case "0":
                    return;
                default:
                    System.out.println("Please choose 0-8.");
            }
        }
    }

    private static List<Integer> readKeys(Scanner in, String prompt) {
        System.out.print(prompt);
        List<Integer> keys = new ArrayList<>();
        if (!in.hasNextLine()) {
            return keys;
        }
        for (String part : in.nextLine().trim().split("[\\s,]+")) {
            if (part.isEmpty()) {
                continue;
            }
            try {
                keys.add(Integer.parseInt(part));
            } catch (NumberFormatException e) {
                System.out.println("    skipped '" + part + "' (not an integer)");
            }
        }
        return keys;
    }
}
```